

CHRIST UNIVERSITY

Department of Computer Science · 5th Semester · CIA-3 Project Development

ShopVerse

Role-Based E-Commerce Management Platform

Node.js · Express.js · MongoDB · Mongoose · JWT · bcrypt

In association with **L&T EduTech**

Team Details

Complete every field below before submission.

#	Name	Register Number	Department	Section
1				
2				
3				
4				

Course: _____ **Semester:** 5th _____

Guide / Faculty in charge: _____

GitHub repository: _____

Date of submission: _____

Table of Contents

1. Problem Statement	3
2. Objectives	3
3. Technology Stack	4
4. System Architecture	4
5. Database Design	5
6. Modules Implemented	7
7. Security Implementation	9
8. Business Rules	10
9. API Reference	11
10. Code Screenshots	13
11. Output Screenshots	17
12. Testing & Results	21
13. Setup Instructions	23
14. Known Limitations	24
15. Conclusion	24

ShopVerse — Project Report

1. Problem Statement

An online marketplace must serve three different kinds of user simultaneously — customers who browse and buy, sellers who list products and fulfil orders, and administrators who oversee the platform. Each role needs a different view of the same underlying data, and none may read or modify what belongs to another.

Systems of this kind fail in predictable ways. Stock oversells when two customers order the last unit at the same moment. Any authenticated user can call any route because role checks were never added. Order status becomes an editable text field, so invalid jumps such as *Delivered* back to *Shipped* pass unnoticed. Coupons are validated in the browser and trusted by the server. A later price change silently rewrites the totals of orders already placed. A single unhandled promise rejection takes the whole server down.

ShopVerse addresses these directly: it is a REST API and web frontend in which the rules that make an e-commerce system *correct* — not merely functional — are enforced on the server and verified by an automated test suite.

2. Objectives

1. Build a secure REST API using JWT authentication and bcrypt password hashing.
2. Enforce role-based access control across Customer, Seller and Administrator roles.
3. Model the catalog, cart, orders, coupons and reviews in MongoDB with deliberate, documented embed-versus-reference decisions.
4. Implement the full order lifecycle as a controlled state machine rather than unrestricted database updates.
5. Keep inventory correct under concurrent access.
6. Validate every incoming request server-side and return consistent, documented JSON errors.
7. Provide seller analytics and administrative reporting using MongoDB aggregation pipelines.
8. Demonstrate the whole system through a Postman collection and a working web frontend.

3. Technology Stack

Layer	Technology	Justification
Runtime	Node.js 22 LTS	Non-blocking I/O suits an API-heavy, I/O-bound workload.
Framework	Express.js 4.21	Minimal and middleware-driven, which maps cleanly onto MVC.
Database	MongoDB 8 (Atlas)	The document model fits orders that embed their own line items.
ODM	Mongoose 8	Schema validation, middleware hooks, population and index declaration.
Authentication	jsonwebtoken	Stateless sessions requiring no server-side session store.
Password hashing	bcryptjs	Adaptive cost factor; 10 rounds configured.
Validation	express-validator	Declarative per-route validation chains.
Security	helmet, cors, express-rate-limit	Security headers, origin control, brute-force protection.
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5	Demonstrates the API live during evaluation.
Charts	Chart.js 4	Seller and administrator analytics.
Testing	Postman / Newman	104 requests and 326 assertions, runnable from the CLI.

Express version note. The project pins Express 4.x. Async controller errors do not auto-forward on version 4, so every asynchronous handler is wrapped in `utils/asyncHandler.js`, which forwards rejections to the centralised error middleware.

4. System Architecture

The application follows an MVC-style layering. A request passes downward through the stack, and any failure at any level is converted by a single error handler into a consistent JSON response.

```

Client (browser frontend / Postman)
|
Routes      Express routers + express-validator chains
|
Middleware  protect (401) → authorize (403) → validate (400)
|
Controllers business logic and business-rule enforcement
|
Models      Mongoose schemas, hooks, virtuals, indexes
|
Database    MongoDB Atlas – 7 collections

↓ any failure at any layer ↓
errorHandler → { success:false, message, errorCode }
```

Directory	Responsibility
<code>config/</code>	Database connection and the constants that define roles, statuses and the transition graph.
<code>models/</code>	Seven Mongoose schemas with validation, hooks, virtuals and indexes.
<code>routes/</code>	Ten routers; each route declares its own auth, role and validation requirements.
<code>controllers/</code>	Ten controllers holding business logic; no SQL-style query building leaks into routes.
<code>middleware/</code>	Authentication, authorization, validation and centralised error handling.
<code>utils/</code>	Typed errors, async wrapper, response envelope, JWT helpers, pagination, seeder.

5. Database Design

Seven collections are used. The governing rule throughout is the one stated in the project brief: **embed** data that is always read together with its parent and rarely updated on its own; **reference** data that is large, shared across many documents, or updated independently.

Collection	Key fields	Relationship	Decision
users	name, email, password, role, addresses[]	1:N products, orders, reviews	addresses embedded
categories	name, slug, parent, icon	self-reference (tree)	reference
products	name, price, category, seller, stock, images[]	N:1 category, N:1 seller	reference
carts	user, items[]	1:1 user	items embedded
orders	orderNumber, user, items[], totals, status	N:1 user	items embedded snapshot
coupons	code, type, value, minOrderValue, expiresAt	1:N orders	reference
reviews	product, user, order, rating, comment	N:1 product, N:1 user	reference

5.1 Why order items are embedded snapshots

This is the most consequential modelling decision in the project. Order items are embedded *and* they copy the product's name, image and price at the moment of checkout, rather than referencing the live product for display.

An order is a **historical record**. If it referenced the product and the seller later raised the price or renamed the item, every past order would appear to change retrospectively — the customer's invoice would no longer match what they paid. Because the item is snapshotted, the order remains a faithful record even if the product is later edited or deactivated. The `product` ObjectId is still stored alongside the snapshot, so the catalog can be reached when it is genuinely needed.

5.2 Field naming

The brief lists *suggested* field names. This implementation uses idiomatic Mongoose naming, because Mongoose conveys the relationship through `ref` and an `Id` suffix becomes misleading after `.populate()`

replaces the id with the document. The mapping is one-to-one.

Brief suggests	This project uses	Reason
passwordHash	password	Field is <code>select:false</code> and hashed by a pre-save hook; it is never anything but a hash.
categoryId, sellerId	category, seller	Declared with <code>ref</code> ; after populate the field holds the document.
parentCategoryId	parent	Reads better on a self-referencing tree.
userId, productId	user, product	Same <code>ref</code> reasoning.
ratingAvg	ratingAverage	Spelled out; paired with <code>ratingCount</code> .
validTill	expiresAt	Matches the <code>startsAt / expiresAt</code> pair and the <code>-At</code> convention used throughout.
discountType	type	Already namespaced by the <code>coupons</code> collection.

5.3 Indexes

Collection	Index	Purpose
users	<code>{ email: 1 }</code> unique	Login lookup and uniqueness guarantee.
products	<code>{ name, description, brand }</code> text	Keyword search without a full regex scan.
products	<code>{ category: 1, isActive: 1 }</code>	The most common listing query.
products	<code>{ seller: 1 }, { price: 1 }, { ratingAverage: -1 }</code>	Seller catalog; price and rating filters and sorts.
categories	<code>{ parent: 1, isActive: 1 }</code>	Building the category tree.
carts	<code>{ user: 1 }</code> unique	Enforces exactly one cart per user.
orders	<code>{ user: 1, createdAt: -1 }</code>	"My orders", newest first.
orders	<code>{ status: 1 }, { items.seller: 1 }</code>	Admin filtering; seller order queue.
reviews	<code>{ user: 1, product: 1 }</code> unique	One review per product per customer, guaranteed by the database.

The final index is worth emphasising: "one review per product" is not merely an `if` statement in a controller — it is a database constraint. Even a race between two simultaneous requests cannot create a duplicate.

6. Modules Implemented

All fourteen mandatory modules are implemented and demonstrable. None is stubbed.

#	Module	Implementation
1	Registration & Authentication	Self-registration for customers and sellers; bcrypt hashing; JWT access and refresh tokens. The admin role is never self-assignable.
2	Role-Based Access Control	<code>protect</code> returns 401; <code>authorize(...roles)</code> returns 403; <code>ownsOrAdmin</code> enforces per-record ownership.
3	Product Catalog Management	Full CRUD scoped to the owning seller, with images, stock, brand and specifications.
4	Category & Sub-Category	Self-referencing hierarchy with a <code>/categories/tree</code> endpoint; deletion refused while in use.
5	Product Search & Filtering	Text-index keyword search plus category, price, rating, brand and availability filters, all combinable, with sorting and pagination.
6	Shopping Cart Management	Add, update, remove and clear, with the requested total validated against live stock on every change.
7	Order Placement & Checkout	Cart converted to an order with an embedded item snapshot, shipping address and computed totals.
8	Order Status Workflow	Placed → Confirmed → Shipped → Delivered, with Cancelled where permitted; governed by a declared transition graph.
9	Inventory & Stock Management	Atomic conditional <code>\$inc</code> reservation; automatic restock on cancellation; low-stock endpoint for sellers.
10	Discount & Coupon Engine	Percentage and flat coupons with maximum-discount caps, minimum-order values, expiry windows and per-user limits.
11	Payment Status Tracking	Payment mode and status recorded per order; mock gateway isolated behind a single function.
12	Reviews & Ratings	Reviews permitted only after delivery; aggregate rating recomputed after every write.
13	Seller Dashboard APIs	Revenue, orders, rating and units-sold metrics with trends; sales analytics; product performance; scoped order queue.
14	Admin Reporting & Analytics	Sales, top-products, user-growth and inventory reports built with aggregation pipelines.

7. Security Implementation

Control	Implementation
Password hashing	bcrypt at cost factor 10, applied in a <code>pre('save')</code> hook guarded by <code>isModified('password')</code> so a profile update never double-hashes.
Password exposure	The field is declared <code>select: false</code> ; it is only ever loaded explicitly during login.
Authentication	Signed JWT access tokens. The user is re-read from the database on every request, so deactivating an account revokes access immediately.
Authorization	Role gates and ownership checks on every write route, producing 403 distinctly from 401.
Account enumeration	Login returns one generic message for both a wrong email and a wrong password.
Input validation	express-validator chains on every write route, with Mongoose schema validation beneath as a second layer.
Injection safety	All queries go through Mongoose with typed schemas; <code>:id</code> parameters are ObjectId-validated before reaching the database.
Secrets management	Connection string and signing keys live only in <code>.env</code> , which is git-ignored; <code>.env.example</code> documents the keys with empty values.
Transport hardening	<code>helmet</code> security headers and a strict CORS origin allowlist.
Brute force	Rate limiting applied to authentication routes per IP.
Error leakage	Stack traces are logged server-side and never returned in production responses.

8. Business Rules

The rules below are what distinguish the project from plain CRUD. Each is enforced on the server and covered by an automated negative test.

Rule	Enforcement	Rejection
Stock can never oversell	Single conditional <code>\$inc</code> with <code>stock: { \$gte: qty }</code> inside the query; a losing race receives <code>null</code> .	400 BUSINESS_RULE_VIOLATION
Cart respects live stock	Requested total re-checked against the product on every add or update.	400
Status follows the graph	Declared transition map; both the source and the target are validated.	400
Limited cancellation window	A customer may cancel only from Placed or Confirmed.	400
Cancellation restores stock	Reverse <code>\$inc</code> when an order leaves the pipeline.	—
Coupons re-validated at checkout	The same <code>evaluate()</code> method runs at preview and at order placement.	400
Review requires delivery	The user must own a Delivered order containing that product.	403 FORBIDDEN
One review per product	Compound unique index on <code>{ user, product }</code> .	409 CONFLICT
Sellers are isolated	Ownership verified before any product write.	403
Category in use	Deletion refused while products or sub-categories reference it.	400

9. API Reference

All endpoints are prefixed with `/api`. Sixty-six routes are mounted in total; the principal ones are listed here, and the complete reference is in `docs/API-Documentation.md` and the exported Postman collection.

Method	Endpoint	Access	Description
POST	/auth/register	Public	Register a customer or seller
POST	/auth/login	Public	Authenticate and issue a JWT
GET	/auth/me	Any role	Current user profile
GET	/products	Public	List, search and filter products
POST	/products	Seller, Admin	Create a product
PUT	/products/:id	Owner, Admin	Update a product
GET	/products/meta/low-stock	Seller, Admin	Low-stock alert list
GET	/categories/tree	Public	Nested category hierarchy
POST	/cart	Customer	Add an item, validated against stock
POST	/coupons/apply	Customer	Validate and apply a coupon
POST	/orders	Customer	Place an order from the cart
PUT	/orders/:id/status	Seller, Admin	Advance the order status
PUT	/orders/:id/cancel	Customer	Cancel where permitted
POST	/payments/:orderId/initiate	Customer	Mock payment
POST	/products/:productId/reviews	Customer	Submit a review after delivery
GET	/seller/dashboard	Seller	Seller metrics and trends
GET	/admin/reports/sales	Admin	Sales summary report
GET	/admin/reports/top-products	Admin	Best-selling products
GET	/admin/reports/user-growth	Admin	Registrations over time

9.1 Response format

Every response uses one envelope. Clients branch on `errorCode`, never on the wording of `message`.

```

SUCCESS 200/201
{
  "success": true,
  "message": "Products fetched.",
  "data": [ ... ],
  "meta": { "page": 1, "total": 33 }
}

ERROR 4xx
{
  "success": false,
  "message": "Coupon has expired.",
  "errorCode": "BUSINESS_RULE_VIOLATION"
}
```

Status	errorCode	Meaning
400	VALIDATION_ERROR	The request itself was malformed.
400	BUSINESS_RULE_VIOLATION	Well-formed, but a business rule rejected it.
401	UNAUTHENTICATED	Missing, malformed or expired token.
403	FORBIDDEN	Authenticated, but wrong role or not the owner.
404	NOT_FOUND	No such record.
409	CONFLICT	Duplicate; violates a uniqueness constraint.

10. Code Screenshots

The excerpts below show the logic behind the modules that carry business rules. The complete set of twenty-two code screenshots is in [docs/code-screenshots/](#).

10.1 Module 1 — Password hashing

Module 1 — Registration & Authentication

backend/models/User.js · lines 78–99

```
User.js

78  * Hash on the way in. The isModified guard is what stops a profile update from
79  * re-hashing an already-hashed password.
80  */
81  userSchema.pre('save', async function hashPassword(next) {
82    if (!this.isModified('password')) return next();
83    const salt = await bcrypt.genSalt(RULES.saltRounds);
84    this.password = await bcrypt.hash(this.password, salt);
85    next();
86  });
87
88  userSchema.methods.matchPassword = function matchPassword(plain) {
89    return bcrypt.compare(plain, this.password);
90  };
91
92  /** Strip sensitive fields from every serialised user. */
93  userSchema.methods.toSafeJSON = function toSafeJSON() {
94    const obj = this.toObject({ virtuals: true });
95    delete obj.password;
96    delete obj.__v;
97    delete obj.id;
98    return obj;
99  };
```

What this shows — Password hashing. The `isModified` guard is what stops a profile update from re-hashing an already-hashed password.

Figure 1. The `isModified` guard is what prevents a profile update from re-hashing an already-hashed password — a common and hard-to-diagnose bug.

10.2 Module 2 — Role-based access control

Module 2 — Role-Based Access Control

backend/middleware/roles.js · lines 5-39

```

roles.js

5  * Route-level role gate. Always mounted AFTER 'protect', so an unauthenticated
6  * request produces 401 (from protect) and an authenticated-but-wrong-role
7  * request produces 403 (from here).
8  */
9  const authorize =
10  (...allowedRoles) =>
11  (req, res, next) => {
12    if (!req.user) return next(ApiError.unauthorized());
13    if (!allowedRoles.includes(req.user.role)) {
14      return next(
15        ApiError.forbidden(
16          `Access denied. This action requires one of: ${allowedRoles.join(', ')}.`
17        )
18      );
19    }
20    next();
21  };
22
23 /**
24  * Ownership check for seller-owned resources. An admin bypasses it; a seller
25  * may only touch their own documents.
26  */
27 const ownsOrAdmin = (getOwnerId) => (req, res, next) => {
28   if (!req.user) return next(ApiError.unauthorized());
29   if (req.user.role === ROLES.ADMIN) return next();
30
31   const ownerId = getOwnerId(req);
32   if (!ownerId || ownerId.toString() !== req.user._id.toString()) {
33     return next(ApiError.forbidden('You can only modify your own resources.'));
34   }
35   next();
36 };
37
38 module.exports = { authorize, ownsOrAdmin };
39

```

What this shows — The 401/403 split: unauthenticated is rejected by protect, wrong-role by authorize, and ownsOrAdmin enforces per-record ownership.

Figure 2. Unauthenticated requests are rejected by `protect` with 401; wrong-role requests by `authorize` with 403; `ownsOrAdmin` adds per-record ownership.

10.3 Module 8 — The order transition graph

Module 8 — Order Status Workflow

backend/config/constants.js · lines 21-42

```
21  * The order lifecycle, expressed as an explicit transition graph.
22  * Anything not listed here is an invalid transition and is rejected with 400.
23  */
24  const ORDER_TRANSITIONS = {
25    [ORDER_STATUS.PLACED]: [ORDER_STATUS.CONFIRMED, ORDER_STATUS.CANCELLED],
26    [ORDER_STATUS.CONFIRMED]: [ORDER_STATUS.SHIPPED, ORDER_STATUS.CANCELLED],
27    [ORDER_STATUS.SHIPPED]: [ORDER_STATUS.DELIVERED],
28    [ORDER_STATUS.DELIVERED]: [],
29    [ORDER_STATUS.CANCELLED]: [],
30  };
31
32  /** States a customer is still allowed to cancel from. */
33  const CUSTOMER_CANCELLABLE = [ORDER_STATUS.PLACED, ORDER_STATUS.CONFIRMED];
34
35  /** Stock is returned to inventory when an order leaves the pipeline. */
36  const RESTOCK_ON = [ORDER_STATUS.CANCELLED];
37
38  const PAYMENT_MODES = ['UPI', 'Card', 'NetBanking', 'COD'];
39
40  const PAYMENT_STATUS = {
41    PENDING: 'Pending',
42    PAID: 'Paid',
```

What this shows — The workflow is a declared transition graph. Anything not listed is an invalid transition and is rejected with 400.

Figure 3. The workflow is declared as data. Any transition not listed is invalid and is rejected, so status can never be set arbitrarily.

10.4 Module 9 — Atomic stock reservation

Module 9 — Inventory & Stock

backend/controllers/orderController.js · lines 20-59

```

20  * this safe: if two checkouts race for the last unit, exactly one matches and
21  * the other gets null back, so stock can never go negative.
22  * Returns the list of products that were successfully reserved.
23  */
24  const reserveStock = async (items) => {
25    const reserved = [];
26    for (const item of items) {
27      const updated = await Product.findOneAndUpdate(
28        { _id: item.product, stock: { $gte: item.quantity }, isActive: true },
29        { $inc: { stock: -item.quantity, numSold: item.quantity } },
30        { new: true }
31      );
32
33      if (!updated) {
34        // Roll back everything already taken, then report which item failed.
35        await releaseStock(reserved);
36        const p = await Product.findById(item.product).select('name stock');
37        throw ApiError.businessRule(
38          p
39            ? `Only ${p.stock} unit(s) of '${p.name}' are available.`
40            : 'One of the products in your cart is no longer available.'
41        );
42      }
43      reserved.push(item);
44    }
45    return reserved;
46  };
47
48  /** Put stock back - used on rollback and on cancellation. */
49  const releaseStock = async (items) => {
50    await Promise.all(
51      items.map((item) =>
52        Product.findByIdAndUpdate(item.product, {
53          $inc: { stock: item.quantity, numSold: -item.quantity },
54        })
55      )
56    );
57  };
58
59  /**

```

What this shows — Stock is reserved with a single conditional \$inc. If the guard fails the update returns null and the order is rejected — two concurrent orders can never oversell.

Figure 4. The stock guard lives inside the query. If two checkouts race for the last unit, exactly one matches and the other receives `null` — stock can never go negative.

10.5 Module 10 — Coupon evaluation

Module 10 — Discount & Coupon Engine
 backend/models/Coupon.js · lines 54-99

```

54  * Single source of truth for coupon eligibility. Returns
55  * { valid, reason, discount } so the caller can surface the exact reason.
56  */
57  couponSchema.methods.evaluate = function evaluate(cartTotal, userId) {
58    const now = new Date();
59
60    if (!this.isActive) return { valid: false, reason: 'This coupon is no longer active.' };
61    if (this.startsAt > now)
62      return { valid: false, reason: 'This coupon is not active yet.' };
63    if (this.expiresAt < now) return { valid: false, reason: 'Coupon has expired.' };
64
65    if (cartTotal < this.minOrderValue) {
66      return {
67        valid: false,
68        reason: 'Minimum order value of Rs. ${this.minOrderValue} required to use this coupon.',
69      };
70    }
71
72    if (this.usageLimit > 0 && this.usedCount >= this.usageLimit) {
73      return { valid: false, reason: 'This coupon has reached its usage limit.' };
74    }
75
76    if (userId && this.perUserLimit > 0) {
77      const timesUsed = this.usedBy.filter(
78        (u) => u.user && u.user.toString() === userId.toString()
79      ).length;
80      if (timesUsed >= this.perUserLimit) {
81        return { valid: false, reason: 'You have already used this coupon.' };
82      }
83    }
84
85    let discount =
86      this.type === COUPON_TYPES.PERCENTAGE
87        ? (cartTotal * this.value) / 100
88        : this.value;
89
90    if (this.type === COUPON_TYPES.PERCENTAGE && this.maxDiscount > 0) {
91      discount = Math.min(discount, this.maxDiscount);
92    }
93
94    // A discount can never exceed the cart itself.
95    discount = Math.min(Math.round(discount), cartTotal);
96
97    return { valid: true, reason: null, discount };
98  };
99
  
```

What this shows — One method decides eligibility and returns the exact reason, so the cart preview and the checkout re-validation can never disagree.

Figure 5. A single method decides eligibility and returns the precise reason, so the cart preview and the checkout re-validation can never disagree.

10.6 Module 12 — Review gating

Module 12 — Reviews & Ratings

backend/controllers/reviewController.js · lines 58-101

```

58  * POST /api/products/:productId/reviews - customers only.
59  *
60  * Business rule: a review requires a DELIVERED order from this user that
61  * contains this product, and only one review per user per product. The unique
62  * compound index is the real guard; the explicit check exists to return a clean
63  * 409 instead of a raw duplicate-key error.
64  */
65  const createReview = asyncHandler(async (req, res) => {
66    const { productId } = req.params;
67    const { rating, title, comment, images } = req.body;
68
69    const product = await Product.findById(productId);
70    if (!product) throw ApiError.notFound('Product not found.');
```

```

71
72    const deliveredOrder = await Order.findOne({
73      user: req.user._id,
74      status: ORDER_STATUS.DELIVERED,
75      'items.product': product._id,
76    }).select('_id');
```

```

77
78    if (!deliveredOrder) {
79      throw ApiError.forbidden(
80        'You can only review a product after your order for it has been delivered.'
81      );
82    }
83
84    const existing = await Review.findOne({ user: req.user._id, product: product._id });
85    if (existing) {
86      throw ApiError.conflict(
87        'You have already reviewed this product. Edit your existing review instead.'
88      );
89    }
90
91    const review = await Review.create({
92      product: product._id,
93      user: req.user._id,
94      order: deliveredOrder._id,
95      rating,
96      title,
97      comment,
98      images: images || [],
99      isVerifiedPurchase: true,
100    });
101
```

What this shows — A review requires a delivered order containing that product. The duplicate check is also a compound unique index, so the database enforces it too.

Figure 6. A review requires a Delivered order containing that product. The duplicate check is backed by a compound unique index.

10.7 Centralised error handling

Non-Functional — Centralised Error Handling

backend/middleware/errorHandler.js · lines 9-64

```

9  * Centralised error handler. Translates the handful of Mongo/Mongoose/JWT
10 * error shapes into proper status codes so nothing leaks out as a raw 500.
11 */
12 // eslint-disable-next-line no-unused-vars
13 const errorHandler = (err, req, res, next) => {
14   let statusCode = err.statusCode || 500;
15   let message = err.message || 'Something went wrong.';
16   let errors = err.details;
17   let errorCode = err.errorCode;
18
19   // Bad ObjectId that slipped past validateObjectId
20   if (err.name === 'CastError') {
21     statusCode = 400;
22     message = 'Invalid value `${err.value}` for field `${err.path}`.';
23     errorCode = ApiError.CODES.VALIDATION_ERROR;
24   }
25
26   // Mongoose schema validation
27   if (err.name === 'ValidationError') {
28     statusCode = 400;
29     message = 'Validation failed.';
30     errors = Object.values(err.errors).map(e => ({
31       field: e.path,
32       message: e.message,
33     }));
34     errorCode = ApiError.CODES.VALIDATION_ERROR;
35   }
36
37   // Unique index violation
38   if (err.code === 11000) {
39     statusCode = 400;
40     const field = Object.keys(err.keyValue || {})[0] || 'field';
41     message = 'A record with this `${field}` already exists.';
42     errorCode = ApiError.CODES.CONFLICT;
43   }
44
45   if (err.name === 'JsonWebTokenError') {
46     statusCode = 401;
47     message = 'Invalid authentication token.';
48     errorCode = ApiError.CODES.UNAUTHENTICATED;
49   }
50   if (err.name === 'TokenExpiredError') {
51     statusCode = 401;
52     message = 'Session expired. Please log in again.';
53     errorCode = ApiError.CODES.UNAUTHENTICATED;
54   }
55
56   // Unexpected failures are logged in full; clients never see the stack.
57   if (statusCode >= 500) {
58     console.error('[ERROR]', err);
59   }
60
61   // Every error response carries a stable machine-readable code alongside the
62   // human message, so clients branch on the code and never on wording.
63   const body = {
64     success: false,

```

What this shows — Every Mongo/Mongoose/JWT error shape is translated to a proper status code and a stable `errorCode`, so nothing escapes as a raw 500.

Figure 7. Every Mongoose, MongoDB and JWT error shape is translated into a correct status code and a stable `errorCode`, so nothing escapes as a raw 500.

11. Output Screenshots

All screenshots were captured from the running application against the live MongoDB Atlas database. The full set of twenty-seven is in [docs/screenshots/](#).

11.1 Home page

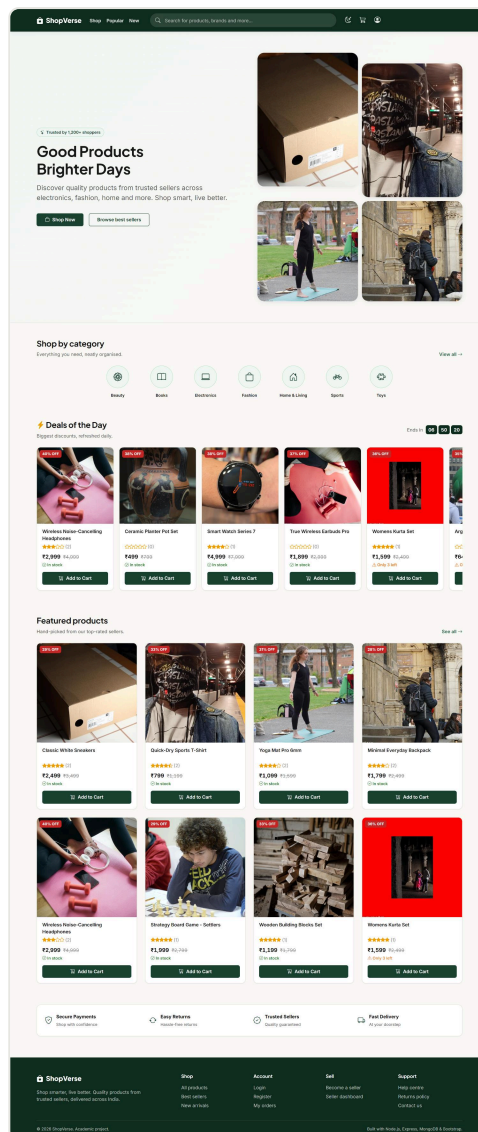


Figure 8. Category navigation, a deals section with a live countdown, and featured products loaded from the API.

11.2 Search and filtering (Module 5)

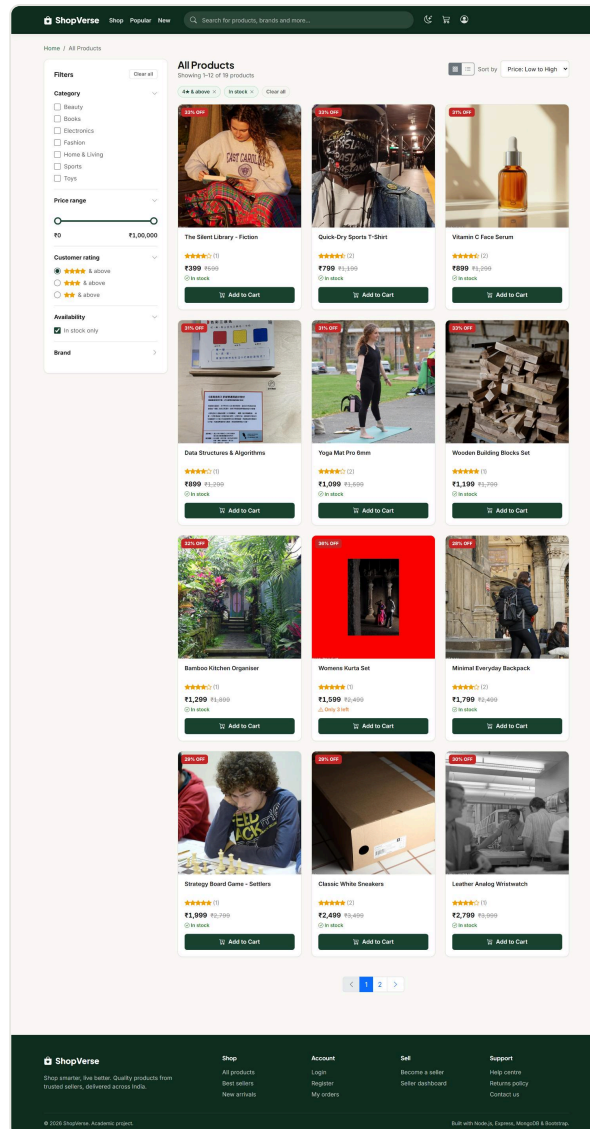


Figure 9. Rating and availability filters applied together and shown as removable chips; the price slider, sort order and view toggle are all combinable.

11.3 Cart and coupon (Modules 6 and 10)

Shopping Cart (3 items)

Item	Quantity	Subtotal
Classic White Sneakers Sold by UrbanStyle ₹2,499 ₹3,499	2	₹4,998
Wireless Noise-Cancelling Headphones Sold by TechWorld ₹2,999 ₹4,999	1	₹2,999

Order Summary

Subtotal (3 items)	₹7,997
Discount on MRP	- ₹4,000
Coupon (WELCOME10)	- ₹500
Delivery	FREE
Total	₹7,497

[Proceed to Checkout →](#)

Have a coupon code?

WELCOME10
You saved ₹500 [Remove](#)

ShopVerse
Shop smarter, live better. Quality products from trusted sellers, delivered across India.

Shop
All products
Best sellers
New arrivals

Account
Login
Register
My orders

Sell
Become a seller
Seller dashboard

Support
Help centre
Returns policy
Contact us

© 2026 ShopVerse. Academic project. Built with Node.js, Express, MongoDB & Bootstrap.

Figure 10. Quantity is capped at live stock, **WELCOME10** has been applied, and all totals are recalculated server-side.

11.4 Order tracking (Module 8)

[Shop](#)
[Popular](#)
[New](#)

[Home](#) / [My Orders](#) / #SV260909QBH7S8

Order #SV260909QBH7S8
 Placed on 09 Sept 2026, 05:10 pm

[Invoice](#)
[Cancel order](#)

Order status
Placed

Placed
 09 Sept 2026, 05:10 pm
 Your order has been placed successfully.

Confirmed
 Pending
 Seller has confirmed your order.

Shipped
 Pending
 Your order is on the way.

Delivered
 Pending
 Your order has been delivered.

Estimated delivery: 14 Sept 2026

Items (2)

Classic White Sneakers
 ₹2,499 × 2

₹4,998

Wireless Noise-Cancelling Headphones
 ₹2,999 × 1

₹2,999

Items total ₹7,997
 Coupon WELCOME10 - ₹500
 Delivery FREE
Total paid ₹7,497
 Payment mode UPI
 Payment status Paid

Need help?
 Questions about this order? Our support team is available 9am–9pm, every day.
[Contact support](#)

Delivery address
Priya Sharma
 123, MG Road, Near Metro Station
 Bengaluru, Karnataka — 560001
 Phone: 9876500021

Shop smarter, live better. Quality products from trusted sellers, delivered across India.

Shop
[All products](#)
[Best sellers](#)
[New arrivals](#)

Account
[Login](#)
[Register](#)
[My orders](#)

Sell
[Become a seller](#)
[Seller dashboard](#)

Support
[Help centre](#)
[Returns policy](#)
[Contact us](#)

 © 2026 ShopVerse. Academic project.
 Built with Node.js, Express, MongoDB & Bootstrap.

Figure 11. The status timeline driven by the transition graph. The cancel action appears only while the rules permit it.

11.5 Seller dashboard (Modules 9 and 13)

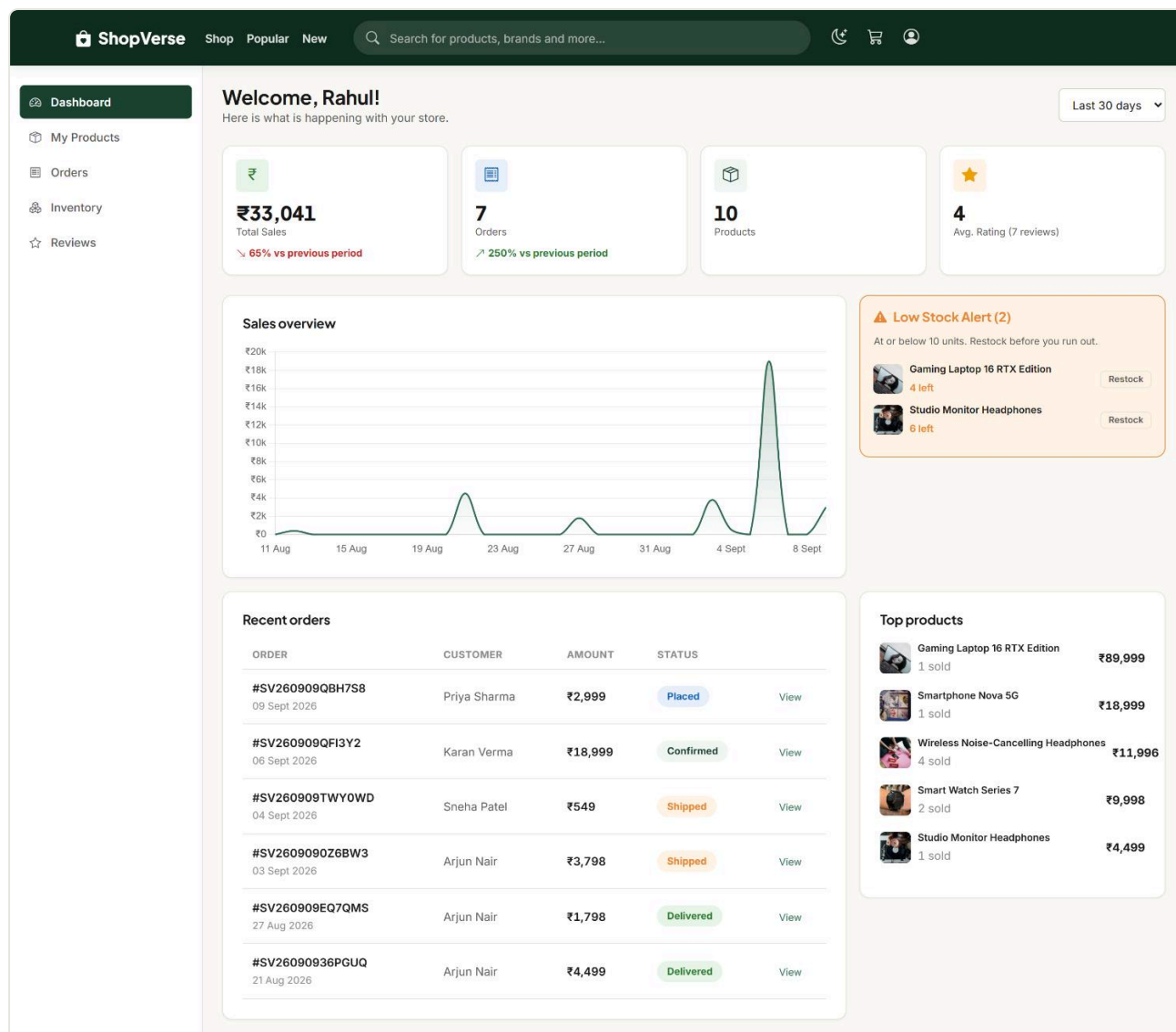


Figure 12. Revenue, order, product and rating KPIs with period-over-period trends, a sales chart, and the low-stock alert panel.

11.6 Admin dashboard (Module 14)

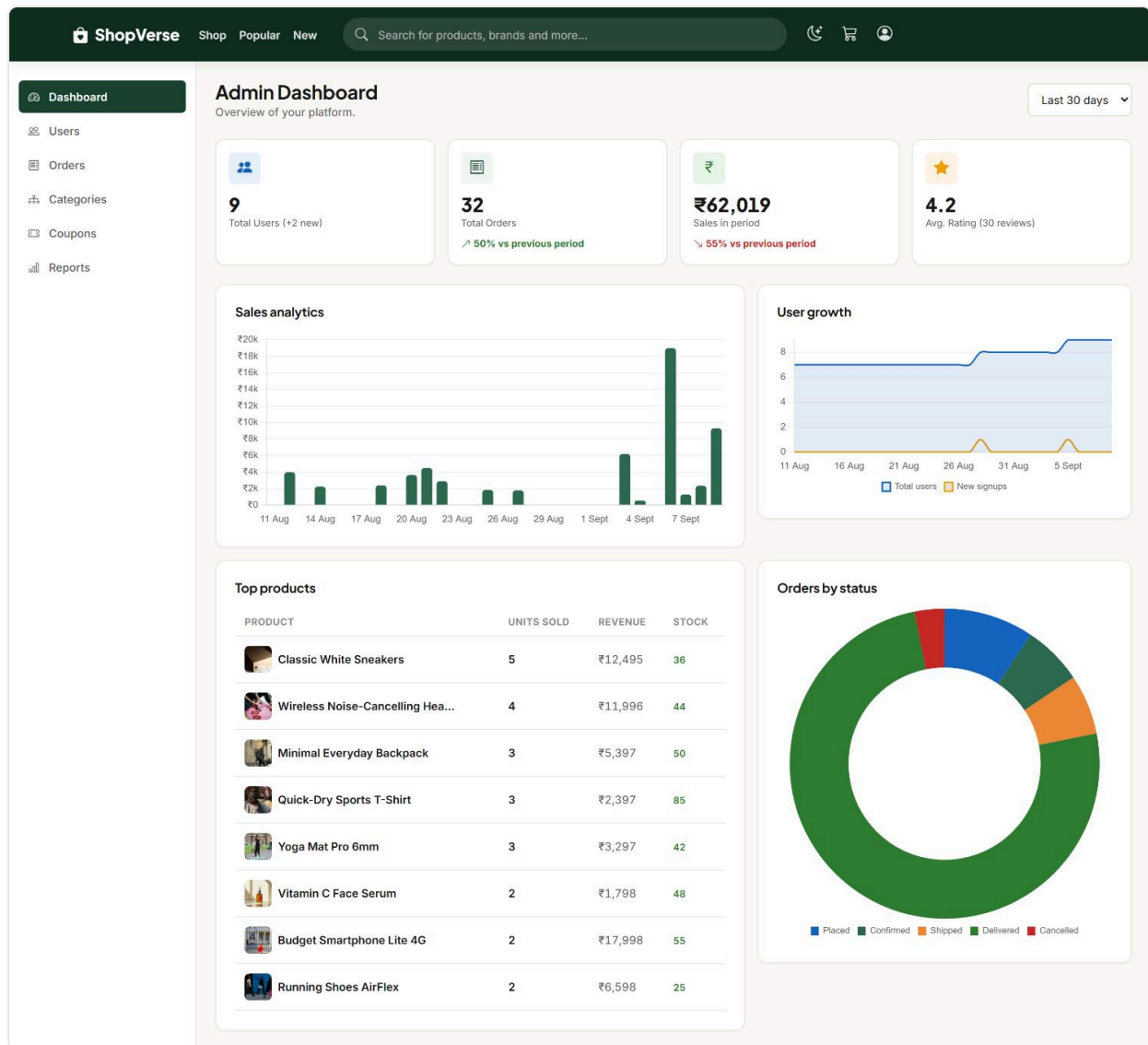


Figure 13. Platform KPIs, sales analytics, the user-growth chart, top products and an orders-by-status breakdown.

11.7 Reviews and ratings (Module 12)

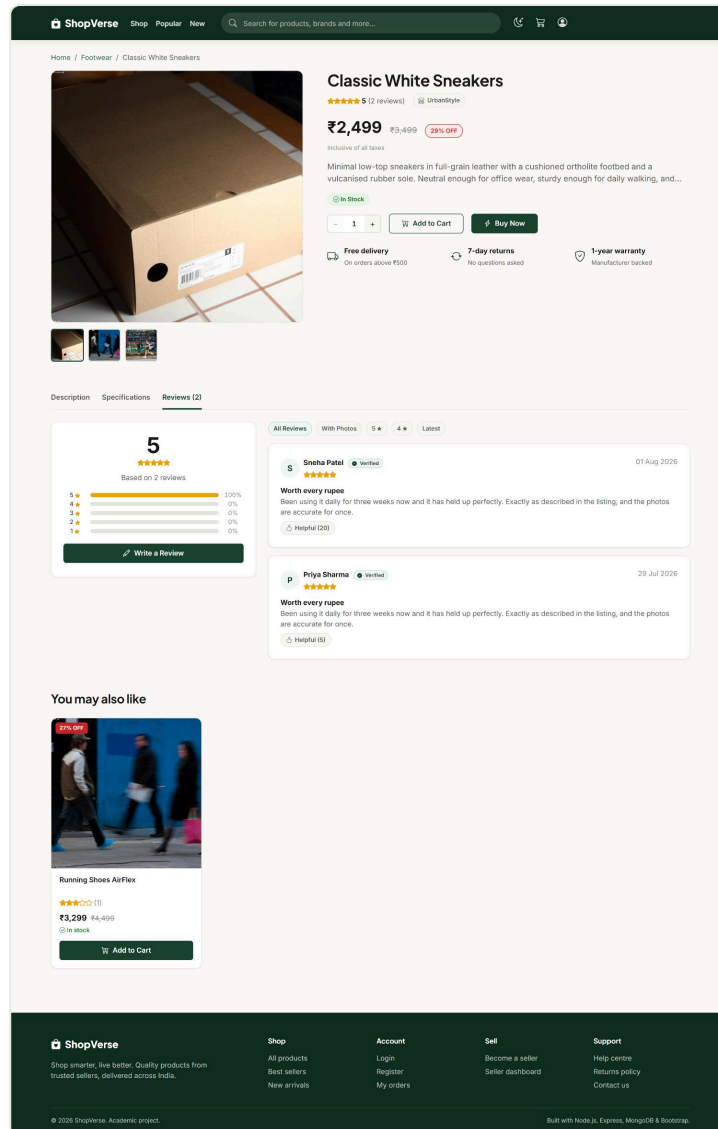


Figure 14. Rating histogram, verified-purchase badges, and filter chips. The review action is disabled for products the user has not received.

12. Testing & Results

The API is covered by a Postman collection of 104 requests carrying 326 assertions, runnable from the command line with Newman. Two assertions run after every request: the response must be JSON, and the status must be below 500.

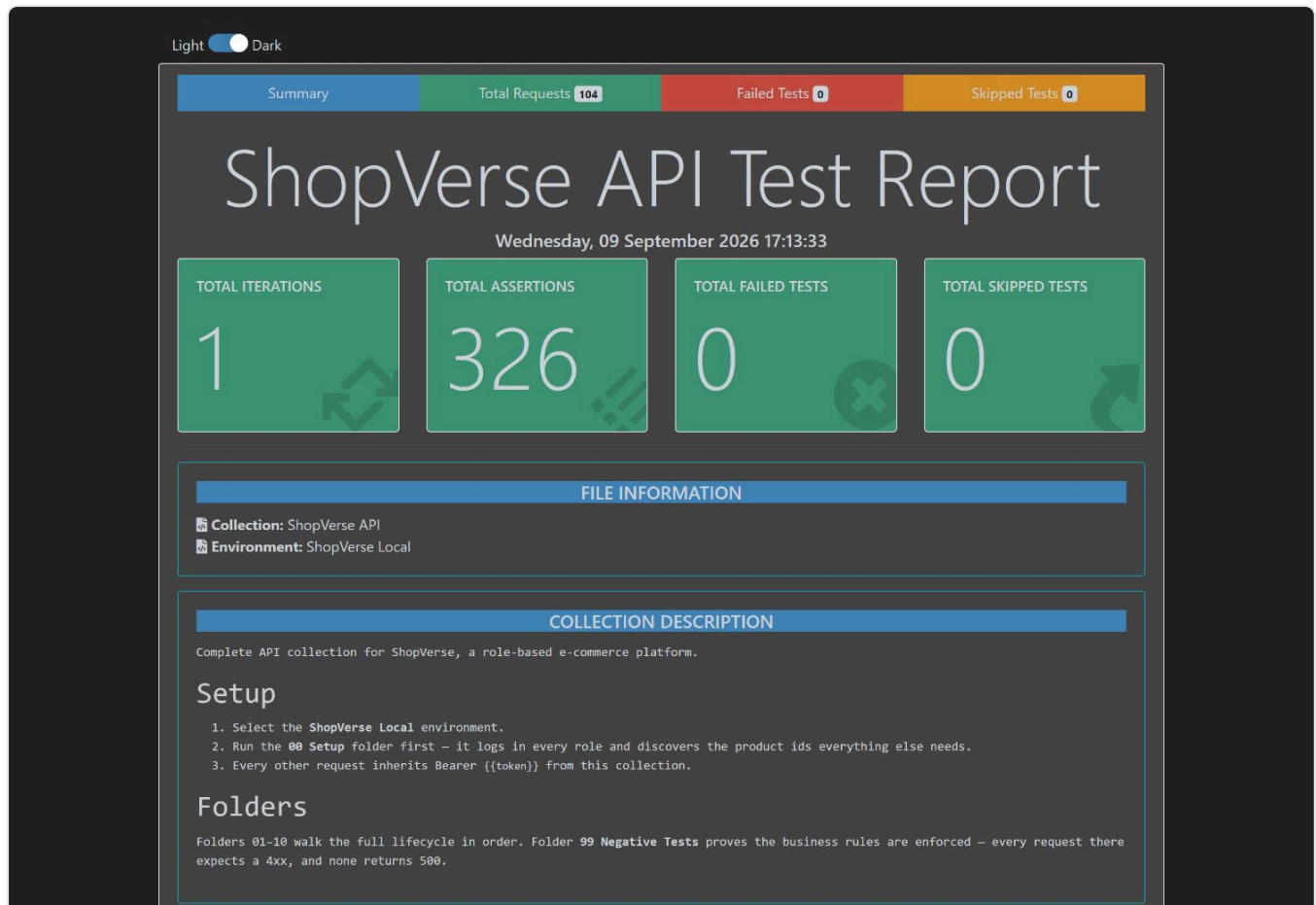


Figure 15. Newman report: 104 requests, 326 assertions, zero failures, executed against MongoDB Atlas.

12.1 Negative testing

The negative-test folder is the part that demonstrates the rules are genuinely enforced rather than merely documented.

#	Test	Expected	Rule proved
1	Protected route with no token	401	Authentication required
2	Malformed or expired token	401	Tokens are verified, not trusted
3	Customer creates a product	403	Role-based access control
4	Seller reads an admin report	403	Role separation
5	Seller edits another seller's product	403	Per-record ownership
6	Registration with a duplicate email	409	Unique index on email
7	Password below minimum length	400	Server-side validation
8	Cart quantity above available stock	400	Stock validation
9	Adding an out-of-stock product	400	Stock validation
10	Applying the expired coupon	400	Coupon expiry
11	Coupon minimum order not met	400	Minimum-order rule
12	Checkout with an empty cart	400	Order precondition
13	Status moved Delivered → Shipped	400	Transition graph
14	Cancelling a delivered order	400	Cancellation window
15	Reviewing a product never purchased	403	Review gating
16	Second review on the same product	409	Compound unique index
17	Malformed ObjectId in the URL	400	Guarded before Mongoose
18	Valid but non-existent id	404	Correct not-found handling

ShopVerse — Security & Business-Rule Enforcement

Live responses captured from the running API. Every failure is deliberate and handled — none returns 500. Unauthenticated requests get 401, authenticated-but-unauthorised get 403.

GET	/cart	401
No token — 401 — A protected route must reject an unauthenticated request.		
<pre>{ "success": false, "message": "Not authenticated. No token provided." }</pre>		
GET	/cart	401
Malformed token — 401 — Tokens are verified, never trusted.		
<pre>{ "success": false, "message": "Invalid authentication token." }</pre>		
POST	/products	403
Wrong role — 403 — A customer may not create products. Authenticated, but unauthorised.		
<pre>{ "success": false, "message": "Access denied. This action requires one of: seller, admin." }</pre>		
GET	/admin/reports/sales	403
Wrong role — 403 — A seller may not read platform-wide admin reports.		
<pre>{ "success": false, "message": "Access denied. This action requires one of: admin." }</pre>		
POST	/cart	400
Out of stock — 400 — Stock is validated before anything reaches the cart.		
<pre>{ "success": false, "message": "Indoor Snake Plant" is currently out of stock." }</pre>		
POST	/coupons/apply	400
Expired coupon — 400 — EXPIRED20 is needed with a past expiry date on purpose.		
<pre>{ "success": false, "message": "Coupon has expired." }</pre>		
POST	/coupons/apply	400
Minimum not met — 400 — BIGONLY requires a ₹10,000 cart.		
<pre>{ "success": false, "message": "Minimum order value of ₹s.10000 required to use this coupon." }</pre>		
GET	/products/abc	400
Invalid ObjectId — 400 — Guarded before Mongoose, so it is a clean 400 and never a 500.		
<pre>{ "success": false, "message": "'abc' is not a valid ID." }</pre>		
GET	/products/50715f77bcf86cd799439821	404
Not found — 404 — A well formed id that matches no document.		
<pre>{ "success": false, "message": "Product not found." }</pre>		

Captured: 9/9/2020, 5:14:57 pm - ShopVerse API - localhost:5000

Figure 16. Live responses for the security and business-rule tests. Critically, none of these returns 500 — every failure mode is handled deliberately.

12.2 Security evidence

ShopVerse — users collection

Read directly from MongoDB Atlas.

Note the password field. Every value is a bcrypt hash beginning `$2a$10$` — the cost factor is 10 and no plaintext password is ever stored.

```
[
  {
    "_id": "6aa14644cb549537e87e770c",
    "name": "Platform Admin",
    "email": "admin@shopverse.com",
    "password": "$2a$10$Ts8KBmWH.ypd6amXN/WtcuY8FwF8o5V/G1FxuNX8qj3Hp6m14neu",
    "role": "admin",
    "isActive": true
  },
  {
    "_id": "6aa14644cb549537e87e7710",
    "name": "Rahul Mehta",
    "email": "seller1@shopverse.com",
    "password": "$2a$10$kqdxjh95wp36EC9JGNHzTujX8BdCr3lCsVOR1fq8Yv9VOEy9FY2vs",
    "role": "seller",
    "isActive": true
  },
  {
    "_id": "6aa14644cb549537e87e7712",
    "name": "Vikram Shah",
    "email": "seller3@shopverse.com",
    "password": "$2a$10$mvvtsoBcFcrIovRT4gef.Da/.H64Bh9zqc8BG3Y5jb58CB6zH/Tm",
    "role": "seller",
    "isActive": true
  },
  {
    "_id": "6aa14644cb549537e87e7711",
    "name": "Ananya Iyer",
    "email": "seller2@shopverse.com",
    "password": "$2a$10$gcMoMSXsG96M/1Q2Xjdo6.51JLejJaGnaFuaKGpbZhF30x3aQqhqi",
    "role": "seller",
    "isActive": true
  },
  {
    "_id": "6aa14645cb549537e87e7719",
    "name": "Arjun Nair",
    "email": "customer2@gmail.com",
    "password": "$2a$10$ASFuVZ.WL/F1rZeqRJ8bROJZzzAYZT3LReQQgzZMrz0DWSiZXEnS6",
    "role": "customer",
    "isActive": true
  }
]
```

Captured 9/9/2026, 5:14:59 pm • MongoDB Atlas • database: shopverse

Figure 17. The users collection read directly from Atlas. Every password is a bcrypt hash beginning `$2a$10$`; no plaintext password is stored anywhere.

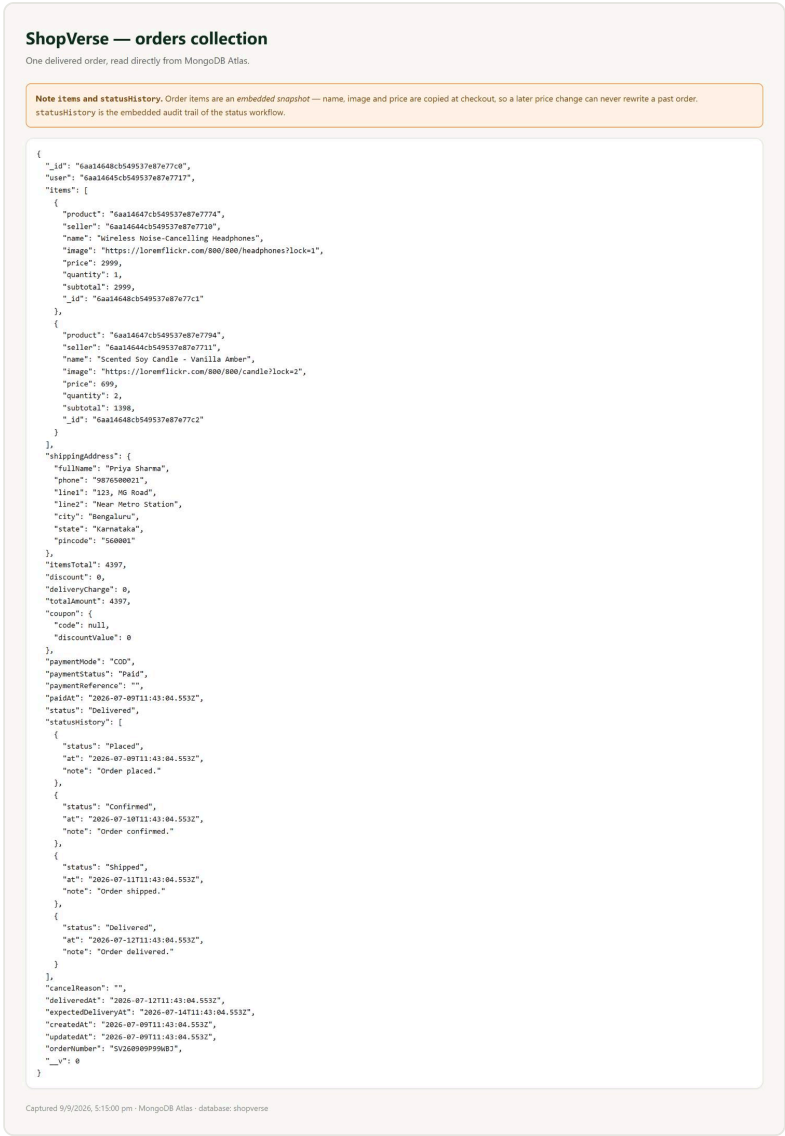


Figure 18. An order document showing the embedded item snapshot and the statusHistory audit trail of the workflow.

12.3 Summary of results

Metric	Result
Mandatory modules implemented	14 of 14
REST endpoints mounted	66
MongoDB collections	7
Postman requests / assertions	104 / 326
Assertion failures	0
Endpoints returning 500 on any tested path	0
Frontend pages	13, responsive, with dark mode

13. Setup Instructions

```
# 1. Clone and install
git clone <repository-url>
cd shopverse-ecommerce-platform/backend
npm install

# 2. Configure environment
copy .env.example .env
# then fill in MONGO_URI, JWT_SECRET and JWT_REFRESH_SECRET
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"

# 3. Seed demo data
npm run seed

# 4. Run
npm run dev # API on http://localhost:5000

# 5. Frontend
# open frontend/index.html with the VS Code Live Server extension
```

Variable	Purpose
MONGO_URI	MongoDB Atlas connection string.
JWT_SECRET / JWT_REFRESH_SECRET	Signing keys; must be different from one another.
BCRYPT_SALT_ROUNDS	bcrypt cost factor (10).
CLIENT_URL	Allowed CORS origin.
LOW_STOCK_THRESHOLD	Drives the seller low-stock alert.
FREE_DELIVERY_ABOVE , DELIVERY_CHARGE	Delivery pricing rules.

13.1 Demo accounts

Role	Email	Password
Administrator	admin@shopverse.com	Admin@12345
Seller	seller1@shopverse.com	Seller@12345
Customer	customer1@gmail.com	Customer@12345

Coupons for the demo. WELCOME10 , FLAT350 and MEGA50 succeed. EXPIRED20 and BIGONLY are seeded to *fail* on purpose — they demonstrate expiry and minimum-order validation during the walkthrough.

14. Known Limitations

The following are out of scope by design, consistent with the boundaries stated in the project brief.

- The payment gateway is mocked; no real transaction is processed.
- No email or SMS notification service is integrated.
- A single currency (INR) and a single time zone are assumed.
- Product images are external URLs rather than uploaded files.
- Social login is not implemented; authentication is a self-built JWT flow.
- Multi-document transactions are not used; stock safety is achieved instead through atomic conditional updates, which is sufficient for the single-document guarantees required here.

15. Conclusion

ShopVerse implements the complete product-to-order lifecycle for three distinct roles on a MongoDB schema whose embed-versus-reference decisions were made deliberately and documented rather than chosen arbitrarily.

What separates it from a CRUD application is that the rules are enforced where they cannot be bypassed. Inventory is correct under concurrency because the stock guard sits inside the update query. The order lifecycle is a declared state machine rather than an editable field. Authorization is checked per role and per record, returning 401 and 403 as distinct outcomes. Coupons and reviews are re-validated on the server and never trusted from the client. Every error, from a malformed ObjectId to a violated unique index, is translated into a consistent JSON response with a stable machine-readable code.

The result is verified rather than asserted: 326 assertions across 104 requests pass against a live Atlas cluster, and no tested path returns a 500.

— End of Report —